

Building Word Embeddings from Scratch

A one-hour technical lesson
Mathematics
Eyob Derese Asefa

How can predicting nearby words teach a network useful word vectors?

Follow one training update from input to changed weights.

The learning path

Represent words

Extract center–context pairs

Predict a context word

Calculate the loss

Derive the gradients

Update E and U

Retrieve trained embeddings

Final embeddings live in E.

The probabilities help train the model. E stores the word vectors we reuse.

One-hot vectors

Vocabulary: [cat, dog, apple, car]

cat = [1, 0, 0, 0]

dog = [0, 1, 0, 0]

apple = [0, 0, 1, 0]

car = [0, 0, 0, 1]

Exactly one position contains 1.
All other positions contain 0.

The 1 identifies the word.

Vector length: $V = 4$

A word ID is an address in a table.

One-hot tells us which word we have. Its coordinates do not encode graded similarity.

Why every distinct pair is equally distant

$$\begin{aligned}\text{cat} \cdot \text{dog} \\ &= 1 \times 0 + 0 \times 1 + 0 \times 0 + 0 \times 0 \\ &= 0\end{aligned}$$

$$\text{cat} \cdot \text{apple} = 0 \text{ too}$$

$$\begin{aligned}\text{distance}(\text{cat}, \text{dog}) \\ &= \sqrt{(1-0)^2 + (0-1)^2} \\ &= \sqrt{2}\end{aligned}$$

Every distinct one-hot pair has this same distance.

These representations cannot express that cat is more related to dog than to car.

Dense embeddings

$V = 1,000$ words

$d = 20$ numbers per embedding

E has shape $1,000 \times 20$.

E contains 20,000 parameters.

$E[\text{cat_id}]$ returns 20 numbers.

Gradient descent changes the trainable values.

We choose d . Training learns the entries.

Coordinates have no predefined labels such as animal or size.

The one-hot vector is conceptual. Efficient code looks up a row without allocating it.

Why a lookup equals $x\mathbf{E}$

Word	E row
the	[0.2, 0.1, 0.4]
cat	[0.8, 0.7, 0.1]
sat	[0.3, 0.2, 0.5]
dog	[0.7, 0.6, 0.2]
mat	[0.1, 0.3, 0.8]

$$\mathbf{x} = [0, 1, 0, 0, 0]$$

$$\mathbf{x}\mathbf{E} = 0 \times \text{row}_0 + 1 \times \text{row}_1 + 0 \times \text{row}_2 + 0 \times \text{row}_3 + 0 \times \text{row}_4$$

$$\mathbf{h} = \mathbf{E}[1] = [0.8, 0.7, 0.1]$$

$$\mathbf{h} = \mathbf{E}[\text{word_to_id}["cat"]]$$

Only the row multiplied by 1 remains. These chosen weights illustrate the mechanism.

Chosen settings and learned values

WE CHOOSE

Vocabulary

Embedding dimension d

Context window

Learning rate η

Number of epochs

TRAINING LEARNS

All entries of E and U

Start with small random values.

One epoch visits every pair once.

Word IDs carry no semantic distance.

For $V = 12$ and $d = 5$, the two matrices contain $2Vd = 120$ trainable numbers.

Context windows create training pairs

Sentence: the cat sat on the mat

Positions: 0, 1, 2, 3, 4, 5

Center = cat at position 1

Window 1: the, sat

Window 2: the, sat, on

Window 2 creates three pairs:

(cat, the)

(cat, sat)

(cat, on)

The second “the” is 3 positions away, outside this window.

The text supplies its own targets

students learn mathematics
students study mathematics

With window 1:
(learn, students), (learn, mathematics)
(study, students), (study, mathematics)

Learn and study predict similar surrounding words.

The text supplies the targets.
No person labels synonyms.

Similar contexts encourage related representations.

Two matrices with different roles

E: input word vectors

Shape $V \times d = 5 \times 3$

Rows represent center words.

$E[\text{cat}] = [0.8, 0.7, 0.1]$

Each word has separate input and output parameters.

U: output context vectors

Shape $d \times V = 3 \times 5$

Columns: the, cat, sat, dog, mat

$[0.1, 0.7, 0.4, 0.6, 0.2]$

$[0.2, 0.8, 0.3, 0.5, 0.1]$

$[0.4, 0.1, 0.7, 0.2, 0.9]$

$U[:, \text{sat}] = [0.4, 0.3, 0.7]$

Example A continues. U maps a d-dimensional input vector to V candidate scores.

Dot products create candidate scores

$$h = [0.8, 0.7, 0.1]$$

$$s_{\text{the}} = 0.8 \times 0.1 + 0.7 \times 0.2 \\ + 0.1 \times 0.4 = 0.26$$

$$s_{\text{sat}} = 0.8 \times 0.4 + 0.7 \times 0.3 \\ + 0.1 \times 0.7 = 0.60$$

$$s_{\text{cat}} = 0.8 \times 0.7 + 0.7 \times 0.8 \\ + 0.1 \times 0.1 = 1.13$$

$$s = [0.26, 1.13, 0.60, 0.85, 0.32]$$

$$s = hU \\ (1 \times 3)(3 \times 5) = (1 \times 5)$$

We read one row of E and use every column of U. Scores are not probabilities.

Softmax normalizes the scores

Word	Score	Probability
the	0.26	0.130588
cat	1.13	0.311701
sat	0.60	0.183469
dog	0.85	0.235579
mat	0.32	0.138663

$$p_j = \exp(s_j) / \sum_r \exp(s_r)$$

Example A continues.
Probabilities sum to 1.
Higher scores give higher relative probabilities.

A score of 0.60 does not mean 60%.

The softmax denominator includes all vocabulary words, even when one word is the target.

Why the probabilities sum to one

We predict the identity of one sampled context token.

That token has exactly one vocabulary value.

Candidates are mutually exclusive and exhaust our fixed vocabulary.

One center can have many neighbors.
(cat, the) is one pair.
(cat, sat) is another pair.

Each pair has its own one-hot target.
A single categorical target identifies one word.

The total probability is 100%, distributed over possible values of one context token.

The derivation: one running example

Vocabulary = [cats, eat, food] Target t = eat

Training pair: (cats, eat)

V = 3 words, d = 2 dimensions

$E[\text{cats}] = h = [1, 0]$

U rows: $[0, 1, -1]$ and $[1, 0, 1]$

h has shape 1×2 .

U has shape 2×3 .

s and p have shape 1×3 .

We use natural logarithms, $\ln$.

All eight steps use these same chosen weights. This is Example C from the guide.

Why a tiny target probability can still underflow

For finite x , $\exp(x) > 0$ For $x < 0$, $0 < \exp(x) < 1$

Extreme example: $s = [1000, 0]$

Observed target: second word

$m = 1000$, shifted scores = $[0, -1000]$

A computer may round $\exp(-1000)$ to 0.

Then computed $p_t = 0$.

Naive calculation: $-\ln(0) = \text{infinity}$

Stable calculation:

$$\begin{aligned} L &= (1000 - 0) + \ln(1 + \exp(-1000)) \\ &\approx 1000 + \ln(1) \\ &= 1000 \end{aligned}$$

Subtracting m prevents overflow. The log-sum-exp loss avoids taking $\ln$ of an underflowed target probability.

Step 8: average the losses over training pairs

$$J(E,U) = (1/N) \sum_{n=1}^N L_n(E,U)$$

Separate four-pair illustration:

cat / sat: $p_t = 0.55$, $L = 0.597837$

dog / barked: $p_t = 0.80$, $L = 0.223144$

cat / the: $p_t = 0.30$, $L = 1.203973$

bird / flew: $p_t = 0.90$, $L = 0.105361$

$N = 4$ pairs

$$\begin{aligned} J &= (0.597837 + 0.223144 \\ &\quad + 1.203973 + 0.105361) / 4 \\ &\approx 0.532578 \end{aligned}$$

Use the same current E and U for every term.

L describes one pair. J describes the average dataset loss. A minibatch averages over its own pairs.

The complete chain to remember

Scores, probabilities, one-pair loss, dataset loss

1 $s = hU$

2 $p_j = \exp(s_j) / \sum_r \exp(s_r)$

3 $y_t = 1$, other $y_j = 0$

4 $L = -\sum_j y_j \ln(p_j) = -\ln(p_t)$

5 $L = -\ln[\exp(s_t) / \sum_r \exp(s_r)]$

6 $L = -s_t + \ln[\sum_r \exp(s_r)]$

7 $L = (m - s_t) + \ln[\sum_r \exp(s_r - m)]$

8 $J = (1/N) \sum_n L_n$

Steps 5–7 rewrite the same loss. Step 7 is the numerically stable implementation.

Why cross-entropy becomes one term

Separate example B

Vocabulary: the, dog, sat, banana, mat

$p = [.15, .20, .55, .01, .09]$

Observed target = sat

$y = [0, 0, 1, 0, 0]$

$$L = -\sum_j y_j \ln(p_j)$$

$$\begin{aligned} L = & -[0 \ln(.15) + 0 \ln(.20) \\ & + 1 \ln(.55) + 0 \ln(.01) \\ & + 0 \ln(.09)] \end{aligned}$$

All zero-weighted terms disappear.

$$L = -\ln(.55) \approx 0.598$$

This assumed probability vector is independent of Example A. $\ln$ denotes natural logarithm.

Why negative log is useful

Probability of observed word

$$L = -\ln(p_{\text{target}})$$

0.90 gives loss 0.105

0.55 gives loss 0.598

0.01 gives loss 4.605

Every score affects the target probability through the denominator.

Lower probability means a larger penalty.

Other candidate scores therefore also receive gradients.

A small loss means the model assigned substantial probability to the observed target.

J measures the average loss

Pair	p_correct	L
cat / sat	0.55	0.598
dog / barked	0.80	0.223
cat / the	0.30	1.204
bird / flew	0.90	0.105

$$J = (1/N) \sum_n L_n$$

$$J \approx (.598 + .223 + 1.204 + .105)/4$$

$$J \approx 0.533$$

Calculate every loss using the same current E and U.

L describes one pair. J describes the average over the dataset.

SGD updates and evaluation measures

TRAINING

$$\theta \leftarrow \theta - \eta \nabla L$$

Compute a gradient for one pair.

Change E and U.

Repeat for the next pair.

$$\nabla J = (1/N) \sum_n \nabla L_n$$

EVALUATION

After an epoch, freeze the weights.

Recompute every pair's loss.

Average these losses to obtain J.

Averaging losses collected while weights changed gives a different metric.

A uniformly sampled pair gradient estimates ∇J . A stochastic step need not reduce full J.

The score gradient is p minus y

$$L = -s_t + \ln(\sum_j \exp(s_j))$$
$$\partial L / \partial s_j = -y_j + p_j$$

$$e = p - y$$

$$p = [0.2, 0.7, 0.1]$$

$$y = [0, 1, 0]$$

$$e = [0.2, -0.3, 0.1]$$

Target: $e_t = p_t - 1$ is negative.

Other words: $e_j = p_j$ is positive.

Descending directly in score space raises the target score and lowers others.

$$\sum_j e_j = 0$$

The target is the second word, index 1 in Python. Shared weights couple the scores.

Gradients for the trainable matrices

$$\begin{aligned}\nabla U &= h^T e && \text{shape } d \times V \\ \nabla h &= e U^T && \text{shape } 1 \times d\end{aligned}$$

$$\begin{aligned}\partial L / \partial U_{kj} &= h_k e_j \\ \partial L / \partial h_k &= \sum_j U_{kj} e_j\end{aligned}$$

Only the selected $E[i]$ row receives the input gradient.

$$\begin{aligned}U &\leftarrow U - \eta \nabla U \\ E[i] &\leftarrow E[i] - \eta \nabla h\end{aligned}$$

Calculate BOTH gradients from the forward-pass weights.
Then change the parameters.
Other rows of E stay unchanged.

The outer product $h^T e$ has the same shape as U . The learning rate η scales each change.

A calculator-sized training example

Example C from the guide

Vocabulary: [cats, eat, food]

$V = 3, d = 2$

Pair: cats / eat

E rows: cats [1, 0]

eat [0, 1]

food [1, 1]

U rows

[0, 1, -1]

[1, 0, 1]

$h = E[\text{cats}] = [1, 0]$

$y = [0, 1, 0]$

Learning rate $\eta = 0.1$

These chosen weights make the arithmetic visible. They are separate from Example A.